MCP сервер для игрового движка Godot

Отчёт по лабораторным работам 1–3

Выполнили: Павлов С.К., Трещев О.Д.

Группа ИДБ-25-15

Преподаватели: Лобанов О.А., Кайкова И.В.

МГТУ «СТАНКИН»

Дисциплина «Основы технологий цифрового производства»

Направление 09.03.03 Прикладная информатика

Москва 2026

Оглавление

1 Введение и постановка задачи

2 Функционал и границы прототипа

3 Аналоги и использование открытого кода

4 Бизнес модель продукта

5 Экономика и привлечение клиентов

6 Варианты использования и организация

7 Архитектура и информационные потоки

8 Контекстная модель IDEF0

9 Декомпозиция цифрового производства

10 Последовательность работ IDEF3

11 Организация и дерево процессов ARIS

12 Событийная цепочка ARIS

13 Разработка и проверка качества

14 Демонстрация и ответы на вопросы

15 Источники и итог работы

Суть проекта

Мы разрабатываем интеграцию, которая связывает ИИ помощника разработчика с движком Godot. Пользователь описывает задачу в Codex или Claude Code, агент изменяет код и вызывает инструменты MCP для работы со сценами, запуска и получения диагностических данных. Результатом становится редактируемый проект игры и отчёт о выполненных действиях.

Продукт предназначен для других разработчиков. Он автоматизирует часть операций цифрового производства: создание и изменение ресурсов, выполнение проверок и передачу обратной связи агенту. Решения о механиках, качестве и выпуске принимает человек. Наша существующая игра служит источником практических требований; отдельный демонстрационный проект используется для проверки MCP.

Что представлено на защите

Подготовлены бизнес модель, процессные схемы и работающий локальный прототип. Прототип основан на Coding-Solo/godot-mcp под лицензией MIT. К 14 унаследованным инструментам добавлены чтение сохранённого дерева сцены и ограниченная проверка проекта. Сквозной тест выполнен на Godot 4.7.1. Коммерческие тарифы и дальнейшие функции описываются как проектируемые.

1 Введение и постановка задачи

Актуальность

При работе с ИИ разработчик часто переносит сообщения об ошибках между движком и чатом, вручную запускает проект и проверяет, что агент изменил нужные сцены. Такой разрыв замедляет итерации и усложняет воспроизведение дефектов. MCP позволяет предоставить ИИ клиенту структурированные инструменты и результаты их выполнения [1–3].

Проблема особенно заметна у начинающих разработчиков и небольших команд, которым одновременно нужны код, сцены, запуск и диагностика. Наличие бесплатных Godot MCP подтверждает существование технического направления, но не доказывает спрос на наш платный продукт. Готовность платить предстоит проверить на пилотах.

Новизна и практическая ценность

MCP и управление Godot через ИИ уже существуют. Наш предполагаемый вклад — поставка понятной интеграции для двух клиентов, единый сценарий изменения и проверки проекта, документация на русском языке и сопровождение. В текущем прототипе вклад конкретизирован двумя новыми инструментами и воспроизводимым тестом. Уникальность на всём рынке и измеренная экономия времени пока не заявляются.

Цель и задачи

Цель — к защите лабораторных работ обеспечить воспроизводимый цикл изменения и проверки проекта Godot без ручного переноса диагностических сообщений между движком и вызывающим клиентом. Критерии достижения: сцена и узел создаются через MCP; сохранённая структура возвращается в

JSON; корректный проект проходит импорт и запуск; некорректные аргументы и ошибка GDScript возвращают отрицательный результат. Снижение затрат времени проверяется отдельно в пилоте и пока не считается измеренным результатом.

Задачи: исследовать аналоги и лицензии; определить интерфейсы инструментов; реализовать базовый цикл работы; подготовить конфигурации клиентов; проверить успешные и ошибочные сценарии; описать цифровое производство и модель продаж; провести пилот с внешними разработчиками.

Объект и предмет исследования

Объект — процессы цифрового производства игр в Godot с участием ИИ агентов. Предмет — способы интеграции ИИ клиента с движком по MCP, организации обратной связи, проверки результата и коммерческой поставки инструментов автоматизации.

2 Функционал и границы прототипа

Таблица 1

Возможности прототипа

Возможность	Текущий статус
Запуск Godot, чтение отладочного вывода, остановка проекта	Унаследовано от Coding-Solo
Создание сцен, добавление узлов, спрайты и ресурсы	Унаследовано; в тесте проверены создание сцены и добавление Label
godot_inspect_scene	Добавлено: JSON с узлами сохранённой .tscn без её инстанцирования
godot_validate_project	Добавлено: импорт, запуск на 1–600 кадров, логи и итог по стадиям
Codex и Claude Code	Подготовлены конфигурации stdio; интерактивное подключение в обоих клиентах ещё не проверено
Живой viewport, управление вводом, откат, биллинг	Не реализовано в нашем расширении; возможные следующие этапы

Роли и сценарий использования

Разработчик задаёт задачу, предоставляет доверенный проект и просматривает изменения. ИИ клиент планирует действия, редактирует код и вызывает инструменты. MCP сервер преобразует вызовы в операции Godot, собирает результат и возвращает его клиенту. Движок импортирует ресурсы, сохраняет сцены и исполняет проект.

Пример: разработчик просит добавить интерфейс результата. Агент изучает проект, создаёт узлы интерфейса, правит скрипт и запускает проверку. При ошибке он получает логи и предлагает исправление. После автоматической проверки разработчик оценивает внешний вид и поведение вручную.

MCP сам не является языковой моделью и не придумывает игру. Он предоставляет инструменты агенту. Работа по естественному языку зависит от подключённого ИИ клиента и его модели. Подписка или API расходы на ИИ оплачиваются пользователем отдельно.

3 Аналоги и использование открытого кода

Таблица 2

Сравнение с исходной базой

Критерий	Исходная база Coding-Solo	Наш прототип
Способ подключения	Локальный MCP сервер	Локальный MCP сервер; сохранён подход базы
Операции со сценой	Создание сцен и добавление узлов	Базовые операции и чтение сохранённой структуры
Проверка проекта	Запуск и отладочный вывод	Импорт и ограниченный запуск с общим статусом
Контроль параметров расширений	Не относится к нашим расширениям	Путь сцены внутри проекта; 1–600 кадров
Воспроизводимость приёмки	Проверяется отдельно	Интеграционный тест и JSON результатов

Сравнение охватывает пять критериев. Coding-Solo выбран исходной базой. Другие подходы представлены ee0pdt/Godot-MCP (сервер и плагин редактора) и mkdevkit/godot-mcp (интеграция с редактором). Их одинаковые локальные испытания не проводились. Преимущество нашей версии относительно взятой базы — отдельный сценарий инспекции сохранённой сцены и проверки с воспроизводимым результатом, а не заявленное превосходство над всем рынком.

Происхождение и собственный вклад

База прототипа — Coding-Solo/godot-mcp, коммит 1209744fad78f3998f98c7394fd0f6ef50da5281. Авторство Solomon Elias и лицензия сохранены. Расширения находятся в src/extensions.ts и src/scripts/godot_inspect.gd. Добавлены интеграционный тест, демонстрационный сценарий и инструкции. Перечень изменений приведён в CHANGES.md.

Лицензия MIT разрешает модификацию и коммерческое распространение при сохранении предусмотренных уведомлений [7]. Мы не выдаём исходную базу за разработку команды. Продажа должна давать покупателю дополнительную ценность: готовые поставки, собственные расширения, обновления и поддержку. Права на сторонние ассеты и код игр рассматриваются отдельно.

Позиционирование

Продукт — интеграция общего назначения для Godot. Начальный набор операций ограничен, но не привязан к одному жанру. Для начала продаж нужен удобный рабочий сценарий и подтверждение пользы, а не обещание автоматически создавать любую коммерчески успешную игру.

4 Бизнес модель продукта

Покупатели и ценностное предложение

Основные сегменты — самостоятельные разработчики Godot и команды из 2–5 человек, уже использующие ИИ инструменты. Дополнительный сегмент — преподаватели и учебные группы. Для новичка ценность заключается в более понятном пути от задания до запуска. Для команды — в единообразной настройке, диагностике и повторяемых проверках.

Пользователь получает локально устанавливаемый пакет, инструкции подключения, примеры и обновления. Код проекта остаётся у пользователя. При этом отправка контекста внешней модели определяется ИИ клиентом: локальное размещение MCP не означает, что все данные остаются на компьютере.

Таблица 3

Предложения и ценовые гипотезы

Предложение	Состав	Цена гипотеза
Community	Открытое ядро, базовые операции, примеры, самостоятельная установка	Бесплатно
Pro	Подготовленные обновления, собственные сценарии автоматизации, приоритетная поддержка в объявленные часы	990 ₽ в месяц за пользователя
Team	До 5 разработчиков, помощь с настройкой, совместимость версий, командные рекомендации	4 990 ₽ в месяц
Внедрение	Разовая настройка под проект и обучение команды	По согласованной смете

Это проверяемые ценовые гипотезы, а не действующий прайс. Оплата, личный кабинет и платные пакеты не реализованы. Лицензия открытого ядра сохраняет права пользователей; платится за дополнительную поставку и

обслуживание. Подписки Codex, Claude и расходы на модели в тарифы не включены.

Ресурсы и каналы поставки

Ключевые ресурсы — компетенции Godot, TypeScript и MCP, тестовые проекты и документация. Поставка предполагается через отдельный GitHub репозиторий и Releases. Сайт может объяснять возможности и принимать заявки; собственная облачная платформа для запуска игр не нужна на первом этапе. До коммерческого запуска необходимо выбрать доступный способ оплаты и оформления продаж.

5 Экономика и привлечение клиентов

Учебный расчёт устойчивости

Для оценки берём только тариф Pro: цена 990 ₽ в месяц, переменные расходы 190 ₽ на активного платящего пользователя. В эту оценку условно входят обслуживание и обработка оплаты. Постоянные расходы приняты равными 50 000 ₽ в месяц: труд 30 000 ₽, сервисы 5 000 ₽, продвижение 10 000 ₽ и резерв 5 000 ₽. Это допущения для модели, а не фактические затраты команды.

Таблица 4

Учебный расчёт экономики

Платящих пользователей	Выручка	Расходы	Результат
25	24 750 ₽	54 750 ₽	−30 000 ₽
50	49 500 ₽	59 500 ₽	−10 000 ₽
100	99 000 ₽	69 000 ₽	30 000 ₽

Вклад одного пользователя: 990 − 190 = 800 ₽. Условная точка безубыточности: округление вверх 50 000 / 800, то есть 63 платящих пользователя. Расчёт не включает налоги, возвраты, стартовую разработку и полную рыночную зарплату основателей. Он показывает чувствительность модели, а не обещает прибыль.

Продвижение

Каналы продвижения: короткие демонстрации рабочего процесса, GitHub, сообщества Godot и учебные мероприятия. В ролике показываются запрос, изменение сцены, проверка и результат. Показатели воронки: переходы на

страницу, установки, первая успешная операция, повторное использование и запрос сопровождения.

Для распространения предлагаются GitHub Releases и техническая документация. Результативность каналов и готовность платить должны проверяться на внешних пользователях; подтверждённых продаж пока нет.

Проверка спроса

Первый пилот: 10 внешних разработчиков, одинаковые задачи и фиксация времени установки, времени до первой операции, ошибок и повторных запусков. Затем — предложение платного пакета с конкретным составом. Нужны фактические оплаты и продления; просмотры роликов и скачивания бесплатного ядра их не заменяют. Игровые альфа тестеры не считаются пользователями MCP.

6 Варианты использования и организация



Рисунок 1 Учебная диаграмма вариантов использования

Граница системы охватывает инструменты интеграции. Разработчик инициирует работу через ИИ клиента, проверяет результат и принимает изменения. Клиент вызывает нужные операции; взаимодействие с внешней моделью находится за границей MCP сервера.

Команда и распределение компетенций

Предлагаемое распределение для нового продукта: Олег Трещев определяет требования к операциям Godot, готовит сценарии, демонстрацию и критерии приёмки. Степан Павлов отвечает за MCP сервер, настройки клиентов, тесты и инструкции. Оба участвуют в анализе обратной связи и развитии бизнес модели. Это план ролей нового проекта, а не утверждение о выполненных каждым человеком коммитах.

Организация проекта состоит из двух участников. Разработка, проверка, документация и сопровождение являются совмещаемыми функциями, а не отдельными штатными отделами.

7 Архитектура и информационные потоки

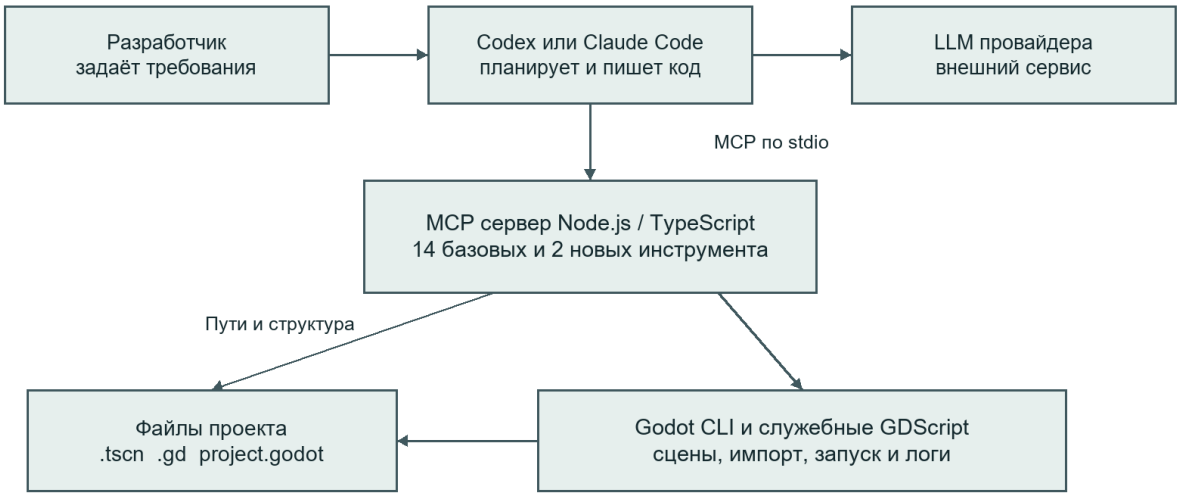


Рисунок 2 Структурно функциональная схема прототипа

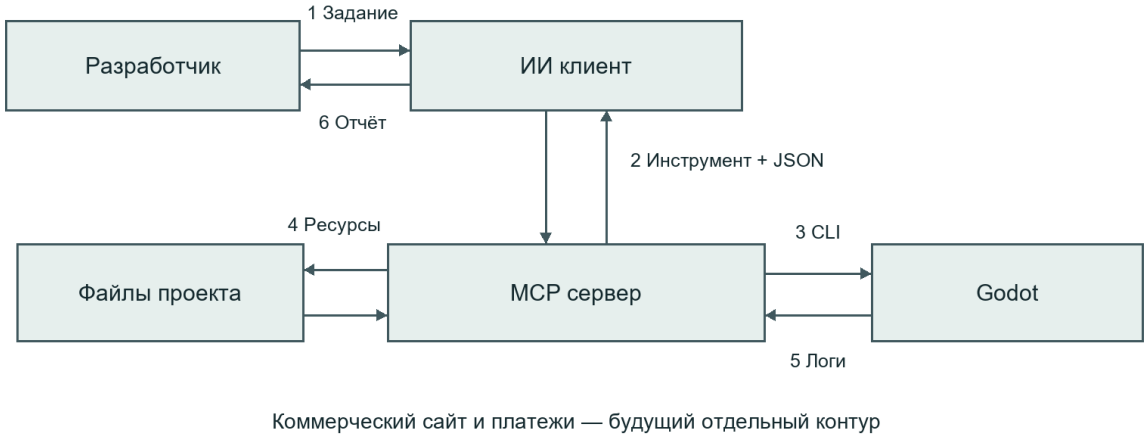


Рисунок 3 Основные информационные потоки

Сервер запускается клиентом как локальный процесс и использует stdio. Godot вызывается отдельным процессом через CLI и служебные GDScript. В текущем прототипе нет постоянно подключённого плагина редактора и визуального онлайн наблюдения. Логи возвращаются вызывающему клиенту; тестовый сценарий сохраняет отчёт в файл [1–3, 8].

8 Контекстная модель IDEF0

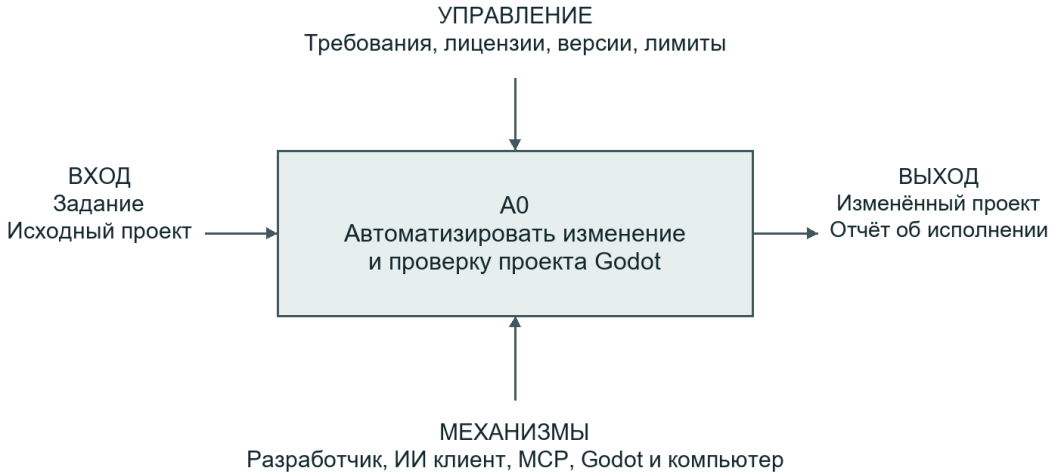


Рисунок 4 Контекстная диаграмма А минус 0 Функция А0

Основная функция — автоматизировать изменение и проверку проекта Godot. Входом являются требования пользователя и исходный проект. Управление задают критерии приёмки, допустимые операции, версии инструментов и лицензии. Механизмами служат разработчик, ИИ клиент, MCP сервер, движок и компьютер. На выходе получаются изменённый проект и результат проверки.

Связь с цифровым производством

Исходные материалы производства — программный код, сцены, настройки и ресурсы. Операции обработки — создание узлов, изменение кода, импорт и запуск. Контроль качества состоит из автоматической диагностики и ручной оценки поведения. Продукт производства — проект или сборка игры. Наш коммерческий продукт предоставляет средства выполнения и контроля этих операций.

Диаграмма описывает целевой рабочий процесс. Прототип покрывает отдельные технические операции; уточнение требований и приёмка пока остаются действиями разработчика и ИИ клиента. Результат с ошибками также допустим как выход диагностики, но не считается готовым продуктом.

IDEF0 используется как учебная нотация функционального моделирования по описанию ICOM в FIPS 183 [9]. Стрелки сверху означают управление, снизу — механизмы; они не задают временную последовательность исполнения.

9 Декомпозиция цифрового производства

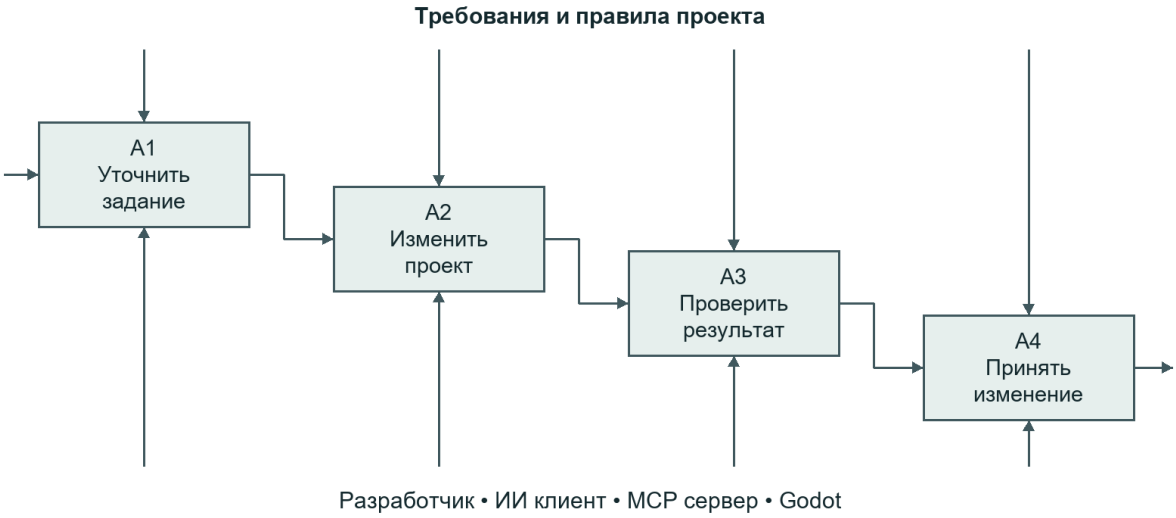


Рисунок 5 Декомпозиция A0 на функции A1–A4

Таблица 5

Функции декомпозиции IDEF0

Функция	Вход и результат	Ответственный
A1 Уточнить задание	Запрос → требования и критерии готовности	Разработчик и ИИ клиент
A2 Изменить проект	Требования и исходники → изменённые сцены и код	ИИ клиент через MCP и файловые инструменты
A3 Проверить результат	Изменённый проект → логи и статус проверки	MCP и Godot; человек для игровых проверок
A4 Принять изменение	Результат и критерии → принятие или задача на исправление	Разработчик

Выход каждой функции используется следующей. Общие требования и правила управляют всеми функциями. При дефекте разработчик или агент уточняет задачу и возвращается к изменению проекта. Публикация коммерческой игры выполняется отдельно после более полного тестирования.

10 Последовательность работ IDEF3

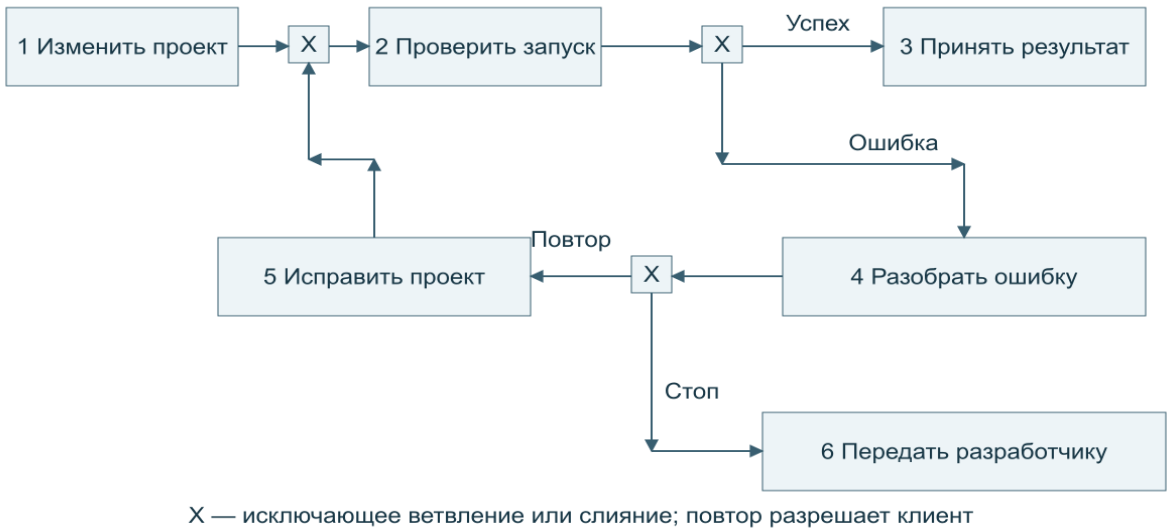


Рисунок 6 Сценарий выполнения и коррекции Блоки UOB и перекрёстки XOR

После изменения проекта выполняется проверка. Успешная проверка ведёт к ручной приёмке. Ошибка запускает диагностику: если исправление возможно в пределах задания, выполняется корректировка и повторная проверка. При исчерпании лимита или неясной причине управление передаётся разработчику.

Перекрёстки X задают взаимоисключающие продолжения. Условие автоматического повтора включает допустимый объём изменений, лимит времени и стоимость работы модели. Для демонстрации достаточно одного исправления и повторного прогона. Сам прототип возвращает диагностический результат; цикл принятия решений исполняет ИИ клиент, а не MCP сервер.

IDEF3 показывает порядок работ в конкретном сценарии, тогда как IDEF0 на предыдущих страницах показывает функции и необходимые им ресурсы. Методические обозначения основаны на IDEF3 Process Description Capture Method Report [10].

11 Организация и дерево процессов ARIS

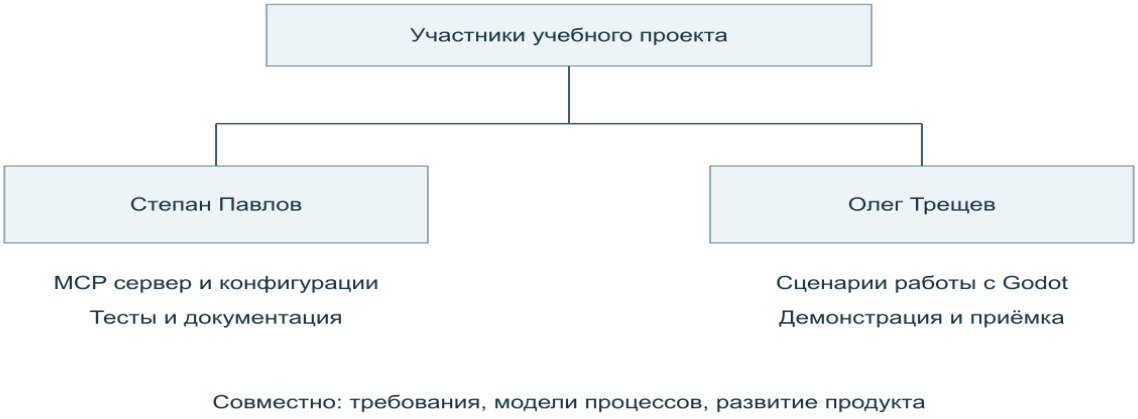


Рисунок 7 Организационное представление команды

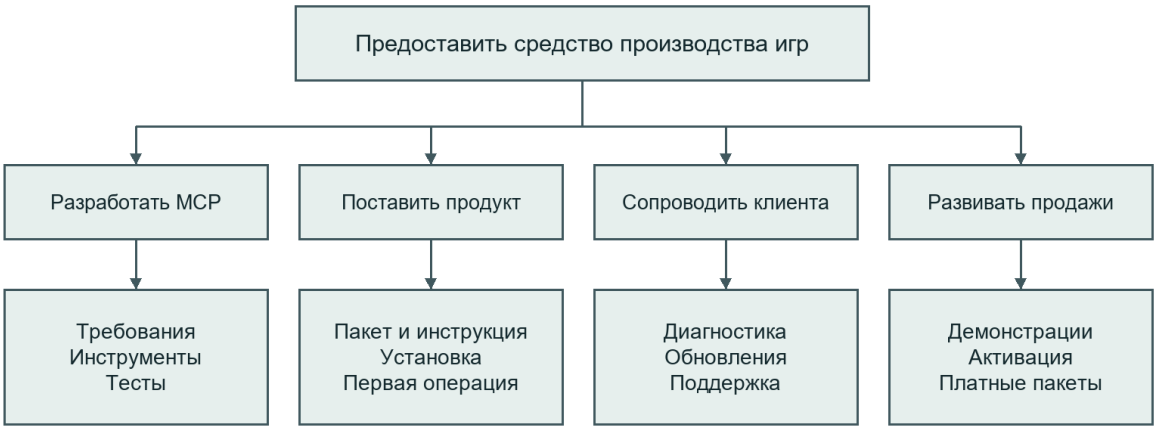


Рисунок 8 Функциональное дерево продукта

В ARIS организационное и функциональное представления дополняют событийную цепочку. Разработка MCP создаёт поставляемую версию; поставка помогает клиенту начать работу; сопровождение удерживает пользователя; продвижение и продажи проверяют коммерческий спрос. Платные функции и поддержка ещё требуют пилота.

12 Событийная цепочка ARIS

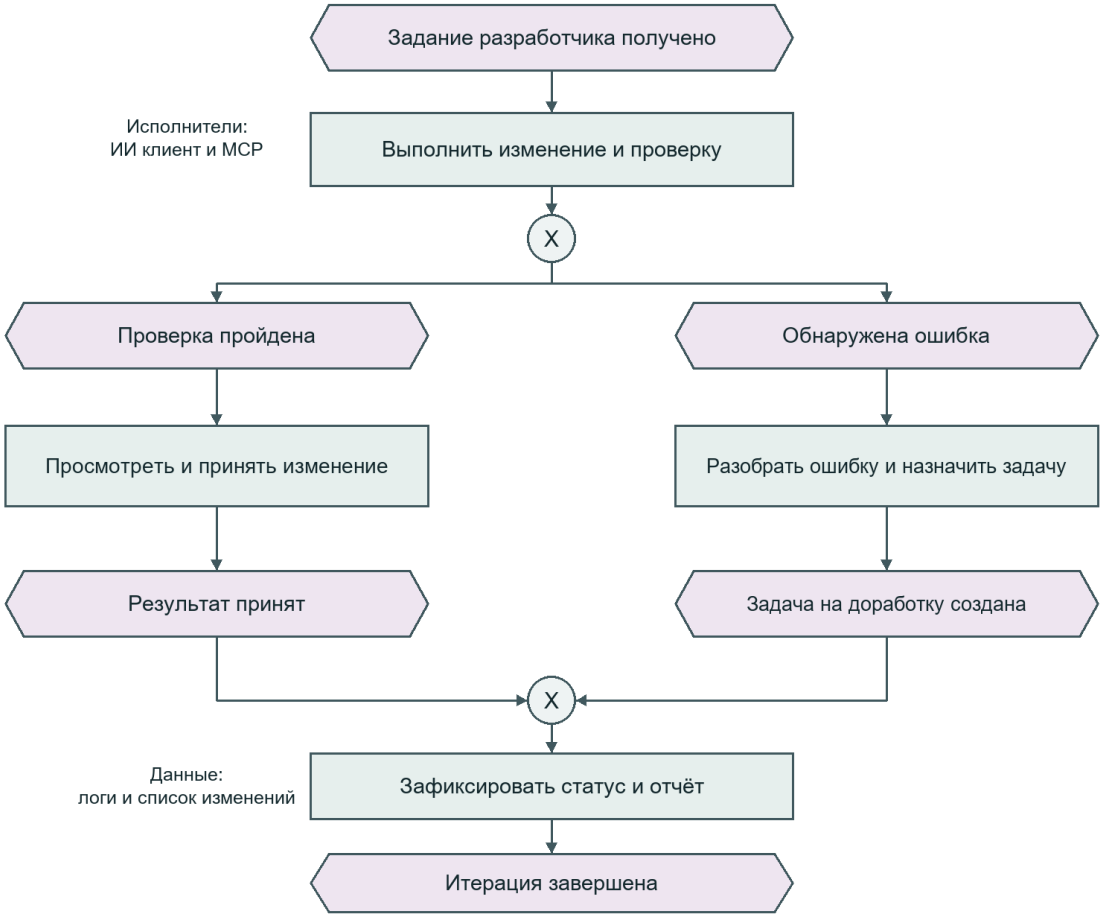


Рисунок 9 Учебная eEPC одной итерации изменения проекта

Шестиугольники — события, прямоугольники — функции, X — исключающее ветвление или слияние. Исполнители и данные поясняют расширенную модель. Каждая ветка завершается фиксацией статуса: принят результат либо создана задача на доработку. Следующая итерация начинается отдельным событием [11].

13 Разработка и проверка качества

Таблица 6

Этапы разработки и проверки

Этап	Проверяемый результат
1 Исследование	Первичные источники, лицензия базы, сценарии пользователей
2 Проектирование	Контракт инструментов, схемы обмена и критерии приёмки
3 Прототип	Отдельная копия исходной базы и собственные расширения
4 Проверка	MCP соединение, работа с реальным Godot, корректный ответ на ошибку
5 Пилот	Установка внешними разработчиками и сравнение выполнения задач
6 Поставка	Версия, совместимость, инструкция, условия поддержки и тарифы

ИИ агенты используются для поиска точек расширения, реализации функций, тестов и документации. Разработчик задаёт границы задачи, читает изменения и принимает результат. История заимствованного кода сохраняется; вклад команды и автоматизированная помощь описываются отдельно.

Проверка прототипа 6 ноября 2026 года

В отдельном тестовом проекте выполнен один интеграционный тест с несколькими проверками: MCP инициализация и список 16 инструментов; создание сцены и добавление Label; чтение дерева с новым узлом; успешный импорт и ограниченный запуск; отказ при неверных параметрах; обнаружение намеренно сломанного GDScript. Отчёт сохранён в evidence/acceptance.json. Демонстрационная сцена создана через MCP и проверена отдельно.

Это техническая проверка минимального сценария на Windows и Godot 4.7.1. Она не доказывает качество всех 14 унаследованных инструментов, безопасность обработки чужих проектов или совместимость со всеми версиями

движка. Подключение и интерактивная работа в Codex и Claude Code — отдельный следующий приёмочный этап.

Риски и меры

Ошибки ИИ уменьшаются ограниченными задачами, проверками и просмотром изменений. Несовместимость версий контролируется матрицей тестов. При запуске доверенного проекта его скрипты могут выполнять действия с правами пользователя; поэтому для демонстрации используется отдельный проект. До распространения нужны обновление зависимостей, аудит унаследованных инструментов и проверка установки на чистой машине.

14 Демонстрация и ответы на вопросы

Порядок демонстрации

1. Показать README, лицензию и перечень собственного вклада. 2. Запустить сборку и тест по инструкции. 3. Выполнить scripts/demo.mjs в новой папке. 4. Показать созданный main.tscn и JSON дерева сцены. 5. Показать успешную проверку и тест со сломанным GDScript. 6. Объяснить, как тот же сервер подключается к Codex или Claude Code по готовым конфигурациям.

Демонстрационный клиент вызывает реальный MCP сервер программно. Он доказывает работу протокола и инструментов, но не изображает разговор с языковой моделью. На защите это следует обозначить прямо. Файлы основной игры для этого сценария не требуются.

Почему это продукт цифрового производства

Покупатель использует интеграцию как инструмент производства собственных игр: она выполняет операции над цифровыми материалами и возвращает результат контроля. Игры являются выходом процесса клиента, а продаваемым продуктом команды становится средство автоматизации этого процесса.

Зачем платить при наличии бесплатных аналогов

За снижение затрат на настройку и сопровождение, собственные рабочие сценарии, совместимые обновления и помощь. Если пользователям достаточно бесплатного ядра, платная гипотеза не подтвердится. Это проверяется пилотом и оплатами, а не числом инструментов.

Что разработано самостоятельно

Два новых инструмента инспекции и проверки, интеграционный тест, демонстрационный сценарий и документация поставки. Базовые 14 инструментов и реализация MCP сервера взяты из открытого Coding-Solo проекта с сохранением лицензии. Новые функции реализованы при помощи ИИ агента и проверены автоматизированным прогоном.

Может ли система создать игру целиком

Целевое развитие позволяет поручать агенту всё больше этапов прототипирования. Текущая версия автоматизирует ограниченный набор операций. Качество дизайна, лицензии ресурсов, ручное тестирование и выпуск остаются ответственностью разработчика. Гарантия коммерческого успеха в продукт не входит.

15 Источники и итог работы

Получен минимальный технический прототип интеграции и модель его превращения в коммерческий продукт. Следующее решение — после пилота определить, за какие сценарии и сопровождение пользователи готовы платить. До этого цены, спрос и преимущество по времени остаются гипотезами.

[1] MCP — разработка сервера
https://modelcontextprotocol.io/docs/develop/build-server

[2] Официальная документация OpenAI — MCP
https://developers.openai.com/codex/mcp/

[3] Claude Code — подключение MCP
https://code.claude.com/docs/en/mcp

[4] Coding-Solo — исходная база
https://github.com/Coding-Solo/godot-mcp

[5] ee0pdt — аналог
https://github.com/ee0pdt/Godot-MCP

[6] mkdevkit — аналог
https://github.com/mkdevkit/godot-mcp

[7] Лицензия исходной базы
https://github.com/Coding-Solo/godot-mcp/blob/main/LICENSE

[8] Godot — командная строка
https://docs.godotengine.org/en/stable/tutorials/editor/command_line_tutorial.html

[9] NIST FIPS 183 — IDEF0
https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/fipspub183.pdf

[10] IDEF3 — описание метода
https://www.idef.com/idef3-process-description-capture-method/

[11] ARIS — событийная цепочка
https://ariscommunity.com/event-driven-process-chain

Методические требования: ЭОС МГТУ «СТАНКИН», курс «Основы технологий цифрового производства», раздел «Лабораторные работы 1–3», задание и шаблон отчёта.
https://edu.stankin.ru/mod/assign/view.php?id=459487. Коммерческие показатели в работе являются учебными допущениями.